

Guia desde cero - Extractor PDF

Instalacion, creacion del proyecto, dependencias, estructura y flujo de ejecucion.

GUIA DESDE CERO - PROYECTO EXTRACTOR PDF CON PYTHON, OCR, IA Y FASTAPI

Objetivo de este documento

Este documento explica, desde cero, como preparar un ordenador Windows para poder ejecutar y entender un proyecto Python como el Extractor PDF.

La idea es que una persona que sabe programar a nivel general, pero que no trabaja habitualmente con Python ni con herramientas de IA, pueda entender:

- que se esta instalando;
- por que se instala;
- como se crea el proyecto;
- como se instalan sus librerias;
- como se organiza el codigo;
- como se ejecuta la API;
- como se prueba la extraccion de datos de un PDF.

El objetivo final es poder llegar a ejecutar en el PC una aplicacion Python con FastAPI que reciba un PDF, extraiga informacion y devuelva un JSON estructurado.

1. Instalacion de Python

1.1. Version de Python elegida

Se va a instalar:

`python-3.10.11-amd64`

Esta version es Python 3.10.11 para Windows de 64 bits.

Se elige Python 3.10 por varios motivos:

- es una version madura y estable;
- tiene muy buena compatibilidad con librerias de IA;
- funciona bien con PyTorch, Transformers, OCR y FastAPI;
- evita problemas que pueden aparecer con versiones demasiado nuevas como Python 3.12 o 3.13;
- muchas librerias de Machine Learning tardan mas en dar soporte completo a las versiones mas recientes de Python.

En proyectos con IA, no siempre conviene instalar "la ultima version" de Python. Muchas veces conviene instalar una version suficientemente moderna, pero muy compatible.

Por eso usamos Python 3.10.

1.4. Pasos para instalar Python

Ejecutar el instalador:

```
python-3.10.11-amd64.exe
```

Durante la instalacion es importante marcar la opcion:

Add python.exe to PATH

Esta opcion permite ejecutar Python desde la terminal de Windows o desde la terminal integrada de PyCharm.

Si no se marca esta opcion, Python puede estar instalado correctamente, pero Windows no sabra encontrarlo al escribir comandos como:

```
python --version
```

o:

```
py -3.10 --version
```

Despues se puede pulsar:

Install Now

Para una instalacion normal no hace falta configurar opciones avanzadas.

1.5. Que instala realmente Python

Al instalar Python se instalan varias piezas:

Python interpreter

Es el programa que ejecuta codigo Python.

pip

Es el gestor de paquetes de Python. Sirve para instalar librerias externas.

Python Launcher

Permite ejecutar versiones concretas de Python con comandos como:

```
py -3.10
```

Libreria estandar

Conjunto de modulos incluidos con Python sin instalar nada adicional.

1.6. Comprobacion de la instalacion

Despues de instalar Python, abrir una terminal de Windows y ejecutar:

```
python --version
```

Si todo esta correcto, deberia aparecer algo similar a:

```
Python 3.10.11
```

Tambien se puede comprobar con:

```
py -3.10 --version
```

Resultado esperado:

```
Python 3.10.11
```

Si estos comandos no funcionan, normalmente significa que Python no se ha anadido al PATH o que Windows no encuentra la instalacion.

1.7. Por que no instalamos las librerias globalmente

Aunque Python instala pip, no es recomendable instalar todas las librerias directamente de forma global en Windows.

Lo correcto es trabajar con un entorno virtual por proyecto.

Un entorno virtual es una carpeta dentro del proyecto que contiene:

- una copia aislada del interprete Python;
- las librerías instaladas solo para ese proyecto;
- las versiones exactas que necesita ese proyecto.

Ventajas:

- un proyecto no rompe otro;
- se pueden tener versiones distintas de librerías en proyectos distintos;
- es mas facil reproducir el entorno en otro PC;
- PyCharm lo gestiona bastante bien.

En Python, ese entorno suele llamarse:

```
.venv
```

2. Instalacion de PyCharm

2.1. Que es PyCharm

PyCharm es un entorno de desarrollo integrado, es decir, un IDE para programar en Python.

En la imagen aparece el instalador:

pycharm-community-2024.3.4

La edicion Community es gratuita y suficiente para este proyecto.

PyCharm ayuda a:

- abrir y organizar el proyecto;
- crear el entorno virtual;
- detectar el interprete Python;
- ejecutar tests;
- abrir una terminal dentro del proyecto;
- navegar entre clases y ficheros;
- instalar librerías;
- detectar errores de sintaxis;
- ejecutar la API FastAPI desde la terminal.

2.2. Por que usamos PyCharm

Se podria programar Python con otros editores, como Visual Studio Code, pero PyCharm es muy comodo para alguien que quiere ver el proyecto de forma ordenada.

Ventajas concretas:

- entiende proyectos Python de forma nativa;
- muestra claramente carpetas, paquetes, clases y tests;
- permite seleccionar el interprete Python 3.10;
- permite crear y usar un entorno virtual;
- tiene terminal integrada;
- facilita ejecutar pytest;

- facilita depurar código paso a paso.

2.3. Instalación de PyCharm Community

Ejecutar el instalador:

pycharm-community-2024.3.4.exe

Durante la instalación se pueden dejar las opciones por defecto.

Opciones útiles si aparecen:

- crear acceso directo en el escritorio;
- asociar carpetas de proyecto con PyCharm;
- añadir PyCharm al menú contextual de Windows;
- reiniciar si el instalador lo solicita.

No es necesario instalar la versión Professional para este caso.

2.4. Diferencia entre PyCharm Community y Professional

PyCharm Community:

- es gratuito;
- sirve para programar en Python;
- permite ejecutar scripts, tests y terminal;
- es suficiente para este proyecto.

PyCharm Professional:

- es de pago;
- incluye soporte más avanzado para desarrollo web, bases de datos, frameworks y herramientas empresariales;
- puede ser útil en otros contextos, pero no es obligatorio para este proyecto.

Para el Extractor PDF, PyCharm Community es suficiente.

2.5. Primera apertura de PyCharm

Al abrir PyCharm por primera vez puede preguntar:

- tema visual;
- plugins;
- configuración inicial;
- privacidad o envío de estadísticas.

No es necesario instalar plugins especiales para empezar.

Lo importante es que PyCharm detecte Python 3.10 o que podamos indicarle manualmente donde está instalado.

2.6. Relación entre Python y PyCharm

Es importante entender que Python y PyCharm no son lo mismo.

Python:

Es el lenguaje y el intérprete que ejecuta el código.

PyCharm:

Es la herramienta visual para escribir, organizar y ejecutar ese código.

PyCharm necesita tener configurado un interprete Python para poder ejecutar el proyecto.
Sin Python instalado, PyCharm puede abrirse, pero no podria ejecutar el codigo del proyecto.

2.7. Comprobacion desde PyCharm

Una vez instalado PyCharm, se puede abrir una terminal dentro del IDE y ejecutar:

```
python --version
```

o:

```
py -3.10 --version
```

Si aparece Python 3.10.11, significa que PyCharm puede encontrar Python correctamente.

Si no aparece, habra que configurar el interprete desde:

Settings / Project / Python Interpreter

En esa pantalla se puede seleccionar:

- Python 3.10 instalado en el sistema;
- o un entorno virtual del proyecto.

3. Creacion del proyecto en PyCharm

Una vez instalado Python y PyCharm, el siguiente paso es crear el proyecto donde vivira el codigo de la aplicacion.

En este caso vamos a crear un proyecto Python normal. No vamos a crear todavia una aplicacion web completa ni una interfaz grafica. El proyecto sera una API que despues podra recibir peticiones desde C#.

3.1. Crear un nuevo proyecto

Abrir PyCharm y seleccionar:

New Project

PyCharm pedira una ubicacion para el proyecto.

Ejemplo:

```
E:\ProyectosPython\ExtractorPDF
```

La ruta puede ser otra, pero conviene que sea una ruta clara y facil de localizar.

Por ejemplo:

```
E:\ProyectosPython\ExtractorPDF
```

Es recomendable evitar rutas dentro de carpetas temporales, descargas o ubicaciones dificiles de recordar.

3.2. Nombre del proyecto

El proyecto se puede llamar:

ExtractorPDF

El nombre debe ser descriptivo. En este caso indica claramente que el proyecto se dedica a extraer informacion de PDFs.

El nombre visible en PyCharm no tiene por que coincidir exactamente con el nombre interno del paquete Python, pero es conveniente que sea claro.

3.3. Seleccionar el interprete Python

En la pantalla de creacion del proyecto, PyCharm preguntara que interprete Python se va a usar.

Debemos seleccionar Python 3.10.

Puede aparecer como:

Python 3.10

o como una ruta parecida a:

```
C:\Users\<usuario>\AppData\Local\Programs\Python\Python310\python.exe
```

Lo importante es que sea Python 3.10 y no otra version distinta.

Si PyCharm no lo detecta automaticamente, se puede buscar manualmente el ejecutable python.exe dentro de la carpeta de instalacion de Python 3.10.

3.4. Crear entorno virtual del proyecto

PyCharm permite crear un entorno virtual al crear el proyecto.

La opcion suele aparecer como:

New environment using Virtualenv

El entorno virtual normalmente se creara dentro del proyecto con el nombre:

```
.venv
```

La estructura quedara parecida a:

```
ExtractorPDF  
.venv ...
```

La carpeta .venv contiene el Python aislado del proyecto y las librerias que instalaremos despues.

Es importante entender que .venv no es codigo de negocio. Es una carpeta tecnica con dependencias del proyecto.

3.5. Por que usamos un entorno virtual

El entorno virtual evita instalar librerias directamente en el Python global del sistema.

Esto es importante porque cada proyecto puede necesitar versiones distintas de librerias.

Ejemplo:

```
Proyecto A  
FastAPI 0.110 PyMuPDF 1.24
```

```
Proyecto B  
FastAPI 0.120 PyMuPDF 1.26
```

Si todo se instala globalmente, un proyecto puede romper otro.

Con .venv, cada proyecto tiene sus propias dependencias.

3.6. Comprobar que el proyecto usa el entorno correcto

Una vez creado el proyecto, abrir la terminal integrada de PyCharm.

Normalmente se abre desde:

View / Tool Windows / Terminal

o desde la pestana inferior:

Terminal

En la terminal deberia verse que el entorno virtual esta activado.

En Windows normalmente aparece algo similar al principio de la linea:

```
(.venv)
```

Ejemplo:

```
(.venv) PS E:\ProyectosPython\ExtractorPDF>
```

Esto significa que los comandos python y pip se ejecutaran dentro del entorno virtual del proyecto.

3.7. Primeros ficheros recomendados

Antes de escribir codigo de negocio, conviene tener algunos ficheros basicos:

README.md

Documento inicial del proyecto. Sirve para explicar que hace el proyecto y como se ejecuta.

requirements.txt

Lista de librerias necesarias para ejecutar el proyecto.

 **src**
 Carpeta donde se ubicara el codigo fuente de la aplicacion.

 **tests**
 Carpeta donde se ubicaran los tests.

 **pdfs**
 Carpeta local de trabajo para dejar PDFs de prueba.

 **docs**
 Carpeta para documentacion del proyecto.

3.8. Estructura inicial esperada

Una estructura inicial razonable seria:

 **ExtractorPDF**
  **.venv**
  **docs**
  **pdfs**
  **src**
  **tests**
  **README.md**
  **requirements.txt**

Todavia no hay logica de extraccion. Solo estamos preparando el contenedor del proyecto.

El codigo real se creara dentro de src.

Los tests se crearan dentro de tests.

Los PDFs de ejemplo se guardaran en pdfs.

4. Instalacion de librerias del proyecto

Una vez creado el proyecto y activado el entorno virtual .venv, el siguiente paso es instalar las librerias que necesita la aplicacion.

En Python, las librerias externas se instalan normalmente con pip.

pip es el gestor de paquetes de Python. Su funcion es descargar e instalar librerias desde el repositorio ofi

4.1. Actualizar pip

El primer comando recomendable es actualizar pip:

```
python -m pip install --upgrade pip
```

Este comando actualiza el instalador de paquetes de Python.

No es obligatorio en todos los casos, pero es recomendable porque:

- evita errores con instalaciones modernas;
- mejora la compatibilidad con paquetes recientes;
- reduce problemas al instalar librerías con dependencias internas.

4.2. Librerías base del proyecto

Para ejecutar la API actual del Extractor PDF se instalarán estas librerías:

fastapi

Framework para crear APIs HTTP en Python. Es la pieza que permite exponer un endpoint para que una aplicación pueda enviar un PDF.

uvicorn[standard]

Servidor ASGI que ejecuta la aplicación FastAPI. FastAPI define la API, pero Uvicorn es quien la levanta en una URL local, por ejemplo `http://127.0.0.1:8000`.

pydantic

Librería para definir modelos de datos y validar estructuras. FastAPI la utiliza internamente, y también se usa para representar entidades como páginas, bloques de texto o páginas renderizadas.

pydantic-settings

Extensión de Pydantic para leer configuración desde variables de entorno o ficheros `.env`. Sirve para centralizar configuraciones del proyecto.

python-multipart

Librería necesaria para que FastAPI pueda recibir archivos subidos por formulario. En este proyecto es importante porque el endpoint `/extract` recibe un PDF como archivo.

pymupdf

Librería para leer y renderizar PDFs. Permite extraer texto embebido, bloques con coordenadas y convertir páginas del PDF a imagen cuando sea necesario.

pillow

Librería de tratamiento de imágenes. Se usa como apoyo cuando trabajamos con OCR o imágenes renderizadas desde PDF.

pytesseract

Wrapper de Python para usar Tesseract OCR. No es el motor OCR en sí, sino la librería que permite llamar a Tesseract desde Python.

httpx

Cliente HTTP usado principalmente por FastAPI/TestClient en tests. Es útil para probar endpoints de la API.

orjson

Librería rápida para trabajar con JSON. Puede ser útil cuando el volumen de datos crece o cuando se quiere optimizar la serialización.

python-dotenv

Permite cargar variables de entorno desde un fichero `.env`. Es útil para configuración local.

pytest

Framework de testing. Permite ejecutar pruebas automáticas para comprobar que el extractor sigue devolviendo los datos esperados.

4.3. Comando de instalación de librerías base

Desde la terminal de PyCharm, dentro del proyecto, se puede ejecutar:

```
pip install fastapi "uvicorn[standard]" pydantic pydantic-settings python-multipart pymupdf pillow pytesseract
```

En Windows PowerShell también suele funcionar:

```
pip install fastapi uvicorn[standard] pydantic pydantic-settings python-multipart pymupdf pillow pytesseract
```

Si PowerShell diera problemas con los corchetes de uvicorn[standard], usar la version con comillas:

```
pip install "uvicorn[standard]"
```

4.4. Importa el orden de instalacion?

Para las librerias base, el orden no es especialmente critico.

pip resuelve dependencias automaticamente. Por ejemplo, FastAPI depende de otras librerias internas, y pip la

Aun asi, como buena practica, el orden recomendado es:

1. Actualizar pip.
2. Instalar librerias base de API y PDF.
3. Instalar librerias de OCR.
4. Instalar librerias de testing.
5. Dejar para mas adelante las librerias pesadas de IA.

El motivo no es que pip lo exija, sino que ayuda a detectar problemas por bloques.

Si algo falla instalando PyMuPDF, sabemos que el problema esta en la parte PDF.

Si algo falla instalando pytesseract, sabemos que el problema esta en la parte OCR.

Si algo falla instalando torch o transformers, sabemos que el problema esta en la parte de IA.

4.5. Diferencia entre pytesseract y Tesseract

Es importante distinguir dos cosas:

pytesseract

Es una libreria Python. Permite llamar a Tesseract desde codigo Python.

Tesseract OCR

Es un programa externo instalado en Windows. Es el motor real que analiza una imagen y obtiene texto.

Instalar pytesseract no instala automaticamente el programa Tesseract.

Por eso, para usar OCR real, tambien hay que instalar Tesseract en Windows.

Normalmente se instala en una ruta parecida a:

```
C:\Program Files\Tesseract-OCR
```

Si Tesseract no esta en el PATH de Windows, Python puede tener pytesseract instalado, pero no encontrara el ejecutable de Tesseract.

4.6. Librerias de IA para una fase posterior

El proyecto actual puede ejecutarse sin instalar todavia las librerias pesadas de IA.

Para una fase posterior, cuando se quiera integrar un modelo de HuggingFace como Qwen u otro LLM local, se necesitaran librerias adicionales.

Estas librerias son:

torch

PyTorch. Es la libreria base para ejecutar muchos modelos de IA.

transformers

Libreria de HuggingFace para cargar modelos de lenguaje, tokenizadores y pipelines.

accelerate

Ayuda a ejecutar modelos de forma mas eficiente y a distribuirlos entre CPU/GPU si aplica.

sentencepiece

Tokenizador usado por algunos modelos.

protobuf

Libreria de serializacion usada internamente por algunos modelos y tokenizadores.

Estas dependencias pueden pesar bastante mas que las librerias base.

Por eso conviene instalarlas solo cuando realmente se vaya a trabajar con modelos de IA.

4.7. Comando de instalacion de librerias IA

Cuando llegue el momento de trabajar con modelos HuggingFace, se puede ejecutar:

```
pip install torch transformers accelerate sentencepiece protobuf
```

En algunos equipos, especialmente si se quiere usar GPU, la instalacion de torch puede requerir un comando especifico segun la version de CUDA.

Para una primera prueba sin GPU, se puede empezar con la instalacion normal.

4.8. Guardar dependencias en requirements.txt

Una vez instaladas las librerias base, conviene guardarlas en el fichero:

requirements.txt

Este fichero permite reproducir la instalacion en otro ordenador.

El contenido base puede ser:

fastapi

uvicorn[standard]

pydantic

pydantic-settings

python-multipart

pymupdf

pillow

pytesseract

httpx

orjson

python-dotenv

pytest

Mas adelante, las dependencias de IA pueden guardarse en otro fichero separado:

requirements-ml.txt

Con contenido:

torch

transformers

accelerate

sentencepiece

protobuf

Separar requirements.txt y requirements-ml.txt tiene una ventaja:

- requirements.txt instala lo necesario para ejecutar la API actual;
- requirements-ml.txt instala solo lo necesario para modelos de IA, cuando haga falta.

4.9. Instalar desde requirements.txt

Si el fichero requirements.txt ya existe, no hace falta escribir todas las librerías a mano.

Se puede instalar todo con:

```
pip install -r requirements.txt
```

Y si mas adelante se quieren instalar las librerías de IA:

```
pip install -r requirements-ml.txt
```

Este sistema es mas comodo y reduce errores al preparar el proyecto en otro ordenador.

4.10. Comprobar que las librerías se han instalado

Para comprobar que la instalacion base funciona, se puede ejecutar:

```
python -c "import fastapi, fitz, PIL, pytesseract, pytest; print('OK')"
```

Si aparece:

OK

significa que Python puede importar las librerías principales.

Nota:

PyMuPDF se instala con el nombre pymupdf, pero en código Python se importa normalmente como:

```
import fitz
```

Por eso en la comprobacion aparece fitz.

4.11. Siguiente paso

Después de instalar las librerías, el siguiente paso será crear la estructura de carpetas y explicar la arquitectura del proyecto.

En el siguiente bloque se explicara:

- que carpetas existen;
- que significa cada capa;
- que son dominio, aplicación, infraestructura e interfaces;
- donde viven los extractores;
- donde viven los tests;
- como se organiza el proyecto para poder crecer.

5. Estructura de carpetas del proyecto

Una vez creado el proyecto e instaladas las librerías, hay que organizar el código.

La organización de carpetas es importante porque este proyecto puede crecer.

Hoy solo extrae datos de un cliente/producto concreto, pero en el futuro podría tener:

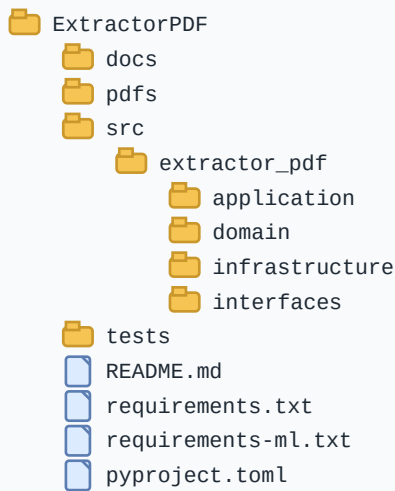
- mas clientes;
- mas tipos de documento;
- mas perfiles de extracción;
- OCR;
- modelos de IA;
- endpoints adicionales;

- contratos configurables desde base de datos.

Por eso conviene evitar tener todo el código mezclado en un único fichero.

5.1. Estructura general

La estructura principal del proyecto es:



Cada carpeta tiene una responsabilidad distinta.

5.2. Carpeta docs

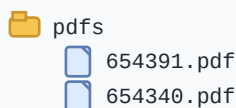
La carpeta docs contiene documentación del proyecto.

Esta carpeta no contiene código de ejecución. Sirve para explicar el proyecto o documentar decisiones.

5.3. Carpeta pdfs

La carpeta pdfs contiene PDFs de prueba.

Ejemplo:



Estos documentos sirven para probar que el extractor funciona correctamente.

En un entorno real, los PDFs llegarán a través de la API, pero durante el desarrollo es muy útil tener PDFs locales para pruebas.

5.4. Carpeta tests

La carpeta tests contiene pruebas automáticas.

Los tests permiten comprobar que el proyecto sigue funcionando después de hacer cambios.

Ejemplos:

```
tests
├── test_api_extraer_raloe_crono.py
├── test_extraer_raloe_crono_actual.py
├── test_extractor_pagina_tecnica.py
├── test_extractor_foso_huida_opciones.py
└── test_extractor_premontada_armario_botonerass.py
```

Los tests son especialmente importantes en este proyecto porque el resultado esperado es un JSON grande. Si se toca un extractor y se rompe un campo, un test puede detectarlo rapidamente.

5.5. Carpeta tests/fixtures

Dentro de tests existe una carpeta de fixtures.

Un fixture es un dato de prueba que se usa como referencia.

Ejemplo:

```
tests
├── fixtures
│   └── expected
│       ├── 654391_full.json
│       ├── 654391_page_5.json
│       ├── 654391_page_6.json
│       ├── 654391_page_7.json
│       ├── 654340_full.json
│       ├── 654340_page_4.json
│       ├── 654340_page_5.json
│       └── 654340_page_6.json
```

Estos JSON representan la verdad esperada.

Es decir:

- se ejecuta el extractor;
- se obtiene un resultado;
- se compara con el JSON esperado;
- si no coincide, el test falla.

Esta estrategia da mucha seguridad cuando el extractor evoluciona.

5.6. Carpeta src

La carpeta src contiene el codigo fuente real de la aplicacion.

Dentro de src se encuentra el paquete principal:

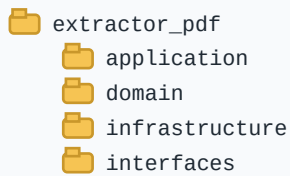
```
src
├── extractor_pdf
```

El paquete extractor_pdf es la aplicacion Python.

Todo el codigo de negocio, API, lectura de PDFs, OCR y extractores vive dentro de este paquete.

5.7. Paquete extractor_pdf

La estructura interna del paquete es:



Esta organizacion sigue una idea parecida a Clean Architecture.

El objetivo es separar responsabilidades:

- domain: conceptos centrales del sistema;
- application: casos de uso;
- infrastructure: detalles tecnicos externos;
- interfaces: puntos de entrada, como la API HTTP.

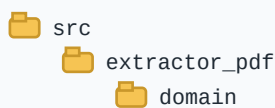
Esta separacion hace que el proyecto sea mas facil de mantener y de ampliar.

6. Capas de la aplicacion

6.1. Capa domain

La carpeta domain contiene las piezas mas basicas del dominio.

Ruta:



Ficheros principales:

entidades.py

puertos.py

6.2. Entidades del dominio

El fichero entidades.py define objetos propios del problema.

Ejemplos:

PaginaPdf

Representa una pagina de un PDF ya leida.

BloqueTexto

Representa un bloque de texto dentro de una pagina, con coordenadas.

PaginaRenderizada

Representa una pagina convertida a imagen.

Estos objetos no son especificos de FastAPI ni de PyMuPDF. Son conceptos internos del proyecto.

Ejemplo conceptual:

Una PaginaPdf tiene:

- numero de pagina;
- texto;
- bloques de texto;

- metodo de extraccion.

Esto permite que el resto de la aplicacion trabaje con PaginaPdf sin preocuparse de como se ha leído el PDF.

6.3. Puertos del dominio

El fichero puertos.py define interfaces o contratos internos.

Ejemplos:

LectorTextoPdf

Define que cualquier lector de PDF debe tener un metodo leer_paginas.

LectorOcr

Define que cualquier lector OCR debe poder leer una pagina.

RenderizadorPaginaPdf

Define que cualquier renderizador debe poder convertir una pagina de PDF en imagen.

ExtractorVisualPagina

Define que cualquier extractor visual debe poder extraer informacion desde una pagina renderizada.

La idea de estos puertos es no depender directamente de una libreria concreta.

Por ejemplo:

Hoy podemos usar PyMuPDF para leer PDFs.

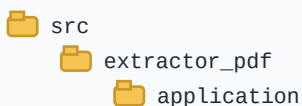
Manana podriamos cambiar a otra libreria.

Si respetamos el contrato LectorTextoPdf, el resto de la aplicacion no tiene que cambiar demasiado.

6.4. Capa application

La carpeta application contiene la logica de aplicacion.

Ruta:



Ficheros principales:

extraer_raloe_crono_actual.py

contrato_vacio.py

campos_extra.py

6.5. Caso de uso principal

El fichero extraer_raloe_crono_actual.py contiene el caso de uso principal actual.

Su responsabilidad es coordinar el proceso completo de extraccion para el perfil Raloe-CRONO actual.

No se encarga de leer directamente el PDF ni de saber detalles internos de PyMuPDF.

Coordina piezas:

- lector de PDF;
- detectores de paginas;
- extractores de secciones;
- contrato vacio;
- campos extra;

- respuesta final.

Conceptualmente hace esto:

1. Recibe bytes de un PDF.
2. Lee las paginas.
3. Comprueba si parece un documento Raloe-CRONO.
4. Detecta las paginas relevantes.
5. Extrae datos de cada pagina.
6. Une todos los datos en un unico JSON.
7. Rellena campos vacios si algo no aparece.
8. Detecta campos extra.
9. Devuelve data y metadata.

6.6. Contrato vacio

El fichero contrato_vacio.py carga el contrato vacio del perfil.

El contrato vacio es un JSON con todas las secciones y todos los campos esperados, pero con valores vacios.

Esto permite cumplir una regla importante:

Si un campo no se encuentra, no se devuelve error. Se devuelve el campo con valor vacio.

Esto es importante para integracion con porque el cliente espera una estructura estable.

6.7. Campos extra

El fichero campos_extra.py detecta campos no previstos.

La idea es:

- el contrato define los campos esperados;
- si aparece un campo nuevo en el formulario;
- y ese campo no esta en el contrato;
- se devuelve dentro de Campos_extra.

Ejemplo:

```
"Campos_extra": {  
  "Micros_de_Freno": "MECANICO"  
}
```

Esto permite detectar cambios en formularios de cliente sin romper la extraccion principal.

6.8. Capa infrastructure

La carpeta infrastructure contiene detalles tecnicos.

Ruta:

```
src  
├── extractor_pdf  
└── infrastructure
```

Aqui vive codigo que depende de librerias concretas o de mecanismos externos.

Ejemplos:

- PyMuPDF;
- Tesseract;
- lectura de PDFs;
- renderizado de paginas;
- detectores concretos;
- extractores concretos de Raloe-CRONO.

6.9. Lectura de PDFs

Ruta:

```
infraestructura
├── pdf
│   ├── lector_pymupdf.py
│   └── renderizador_pymupdf.py
```

```
lector_pymupdf.py
├── Lee el PDF y lo convierte en una lista de PaginaPdf.
└── Extrae texto embebido y bloques con coordenadas.
```

```
renderizador_pymupdf.py
├── Convierte una pagina del PDF en imagen.
└── Esto es util para OCR.
```

PyMuPDF es la libreria concreta usada para esta tarea.

6.10. OCR

Ruta:

```
infraestructura
├── ocr
│   ├── lector_ocr_nulo.py
│   └── extractor_visual_tesseract.py
```

```
lector_ocr_nulo.py
├── Es un lector OCR que no hace nada.
└── Se usa cuando no queremos pasar OCR automaticamente.
```

```
extractor_visual_tesseract.py
└── Usa pytesseract para extraer texto desde una imagen.
```

Esta parte permite que el proyecto pueda evolucionar hacia PDFs escaneados o imagenes.

6.11. Detectores de paginas

Ruta:

```
infraestructure
├── selection
│   └── detectores_paginas_raloe_crono.py
```

Este fichero contiene detectores de paginas relevantes.

Actualmente existen detectores para:

- pagina tecnica;
- pagina de foso, huida y opciones;
- pagina de premontada, armario y botoneras.

El objetivo de estos detectores es no asumir que los datos estan siempre en la misma pagina fisica.

Por ejemplo:

En un PDF la pagina tecnica puede ser la pagina 5.

En otro PDF la pagina tecnica puede ser la pagina 4.

Por eso se buscan senales de contenido en lugar de asumir siempre "las tres ultimas paginas".

6.12. Extractores Raloe-CRONO

Ruta:

```
infraestructure
├── extraction
│   └── client_base
│       ├── extractor_pagina_tecnica.py
│       ├── extractor_foso_huida_opciones.py
│       ├── extractor_premontada_armario_botonerasy.py
│       ├── extractor_observaciones_resumen.py
│       └── raloe_crono_empty_contract.json
```

Cada extractor se encarga de una parte del documento.

```
└── extractor_pagina_tecnica.py
    └── Extrae datos generales, normas, caracteristicas, traccion y puertas.
```

```
└── extractor_foso_huida_opciones.py
    └── Extrae gestion de foso, huida reducida y opciones.
```

```
└── extractor_premontada_armario_botonerasy.py
    └── Extrae premontada, armario, botonera exterior y botonera de cabina.
```

```
└── extractor_observaciones_resumen.py
    └── Extrae observaciones desde la pagina resumen.
```

```
└── raloe_crono_empty_contract.json
    └── Define la estructura completa esperada del JSON para Raloe-CRONO con valores vacios.
```

6.13. Capa interfaces

La carpeta interfaces contiene los puntos de entrada a la aplicacion.

Ruta:

```
src
├── extractor_pdf
│   └── interfaces
│       └── api
```

Actualmente el punto de entrada principal es una API FastAPI.

Ficheros:

main.py

dependencias.py

6.14. API FastAPI

main.py define la aplicacion FastAPI.

Actualmente expone:

GET /health

Endpoint simple para comprobar que la API esta viva.

POST /extract

Endpoint principal. Recibe un PDF y devuelve el JSON extraido.

dependencias.py crea el caso de uso principal y conecta las piezas necesarias.

Esto separa la API de la logica de extraccion.

La API no deberia saber todos los detalles internos de como se extraen campos. Solo recibe el archivo, llama al caso de uso y devuelve la respuesta.

7. Resumen de responsabilidades

Resumen rapido:

```
domain
└── Define conceptos e interfaces.
```

```
application
└── Coordina casos de uso.
```

```
infrastructure
└── Implementa detalles tecnicos concretos.
```

```
interfaces
└── Expone la aplicacion al exterior.
```

```
tests
└── Verifica que todo funciona.
```



pdfs



Contiene documentos de prueba.



docs



Contiene documentacion.

7.1. Por que esta separacion es importante

Esta separacion permite que el proyecto crezca sin convertirse en un unico fichero dificil de mantener.

Ejemplos:

Si manana cambiamos PyMuPDF por otra libreria:

Se tocaria principalmente infrastructure/pdf.

Si manana cambiamos la forma de detectar paginas:

Se tocaria infrastructure/selection.

Si manana anadimos otro cliente:

Se anadirian nuevos extractores o perfiles.

Si manana C# llama a la API:

La entrada seguira estando en interfaces/api.

Si manana se anade un LLM:

Se podria anadir como servicio dentro de infrastructure o application, sin romper el resto.

7.2. Siguiete paso

Una vez explicada la estructura, el siguiente paso sera explicar el flujo de ejecucion real:

- como se levanta la API;
- que ocurre al llamar a /extract;
- como se detectan paginas;
- como se extraen datos;
- como se genera el JSON final;
- como se prueban los resultados con pytest.

8. Flujo completo de ejecucion

En este punto ya tenemos instalado Python, creado el proyecto, instalada la librerias y explicada la estructura de carpetas.

Ahora vamos a explicar que ocurre desde que se arranca la aplicacion hasta que se obtiene el JSON final.

La idea general es esta:

1. Arrancamos la API con FastAPI.
2. Abrimos Swagger en el navegador.
3. Subimos un PDF.
4. La API recibe el PDF.
5. El caso de uso lee todas las paginas.
6. Se comprueba si el PDF parece un Raloe-CRONO.
7. Se detectan las paginas importantes.
8. Se extraen los datos de cada pagina.

9. Se completan los campos no encontrados con valor vacío.
10. Se detectan posibles campos extra.
11. Se devuelve un JSON final.

8.1. Arrancar la API

La aplicación se arranca desde la terminal del proyecto.

Primero hay que estar dentro de la carpeta del proyecto:

```
E:\ProyectosPython\ExtractorPDF
```

Después se activa el entorno virtual si no está activo:

```
.venv\Scripts\activate
```

Cuando el entorno virtual está activo, normalmente se ve algo parecido a esto al principio de la línea de terminal:

```
(.venv)
```

Para levantar la API se ejecuta:

```
uvicorn extractor_pdf.interfaces.api.main:app --reload
```

Explicación del comando:

uvicorn

Es el servidor que ejecuta la aplicación FastAPI.

extractor_pdf.interfaces.api.main

Es la ruta del fichero Python donde está definida la API.

app

Es la variable FastAPI creada dentro de main.py.

--reload

Hace que el servidor se reinicie automáticamente cuando se modifica código. Es útil durante el desarrollo.

Si todo ha ido bien, la terminal mostrará una URL parecida a esta:

```
http://127.0.0.1:8000
```

8.2. Abrir Swagger

FastAPI genera automáticamente una página de pruebas llamada Swagger.

Se abre en el navegador con esta dirección:

```
http://127.0.0.1:8000/docs
```

Swagger permite probar la API sin tener que crear todavía un cliente.

En esta pantalla aparecen los endpoints disponibles:

GET /health

Sirve para comprobar que la API está arrancada.

POST /extract

Sirve para enviar un PDF y obtener el JSON extraído.

8.3. Comprobar que la API está viva

Antes de subir un PDF conviene probar el endpoint:

GET /health

Si funciona, devuelve:

```
{  
  "status": "ok"  
}
```

Esto no extrae ningun dato.

Solo confirma que FastAPI esta funcionando y que el servidor responde.

8.4. Enviar un PDF a /extract

El endpoint principal es:

POST /extract

Este endpoint recibe:

file

El archivo PDF.

profile_id

El perfil de extraccion. Actualmente el valor soportado es:

raloe_crono

En Swagger se pulsa sobre POST /extract, despues sobre Try it out, se selecciona el PDF y se ejecuta la llamada.

En una fase futura, este mismo endpoint sera llamado desde una API o aplicacion.

La idea sera la misma:

- Aplicación envia el PDF.
- Python lo procesa.
- Python devuelve un JSON.
- Aplicacion consume ese JSON y lo transforma en sus clases o entidades.

8.5. Entrada real en el codigo

La entrada esta en:

src/extractor_pdf/interfaces/api/main.py

Dentro de ese fichero esta el endpoint:

POST /extract

De forma resumida, hace esto:

1. Recibe el archivo PDF.
2. Comprueba que profile_id sea raloe_crono.
3. Lee los bytes del PDF.
4. Crea el caso de uso de extraccion.
5. Ejecuta la extraccion.
6. Anade metadatos como filename y profile_id.
7. Devuelve la respuesta.

Es importante entender que main.py no contiene la logica de extraccion.

main.py solo actua como puerta de entrada.

La logica real esta en application e infrastructure.

8.6. Creacion del caso de uso

El endpoint usa este fichero:

src/extractor_pdf/interfaces/api/dependencias.py

Este fichero crea el caso de uso:

CasoUsoExtraerRaloeCronoActual

Y le inyecta el lector de PDF:

LectorTextoPyMuPdf

Ahora mismo se usa:

LectorOcrNulo

Esto significa que el flujo principal actual esta preparado para trabajar con texto extraido directamente del PDF.

La parte OCR existe en el proyecto, pero no se esta usando como lectura principal del endpoint actual.

Esto es intencionado: si el PDF ya contiene texto, es mas rapido y mas limpio extraerlo directamente que pasar siempre por imagen y OCR.

Mas adelante, si llegan PDFs que son solo imagen, se puede cambiar la dependencia para usar OCR o una estrategia combinada.

8.7. Lectura de paginas

El caso de uso recibe los bytes del PDF.

El primer paso interno es leer las paginas:

```
paginas = self.lector_pdf.leer_paginas(bytes_pdf)
```

Cada pagina se convierte en una entidad del dominio:

PaginaPdf

Una PaginaPdf contiene:

numero

Numero de pagina dentro del documento.

texto

Texto extraido de esa pagina.

bloques

Bloques de texto con informacion de posicion.

Los bloques son importantes porque en muchos formularios no basta con saber que texto hay.

Tambien importa donde esta colocado.

Por ejemplo, hay campos que no tienen una etiqueta clara, pero siempre aparecen cerca de otro campo o en una zona concreta del formulario.

8.8. Comprobar si el PDF es Raloe-CRONO

Antes de intentar extraer todos los campos, el sistema comprueba si el PDF parece pertenecer al perfil Raloe-CRONO.

Esto evita devolver datos falsos cuando se sube un PDF que no corresponde.

La comprobacion busca senales como:

- CRONO
- MANIOBRA CARLOS SILVA
- Traccion Electrica
- Gestion foso / huida reducida
- Premontada

No hace falta que aparezcan todas.

La idea es tener suficientes pistas para considerar que el documento pertenece al perfil correcto.

Si no se detectan suficientes senales, la API no lanza error.

Devuelve un JSON con:

is_raloe_crono: false

status: not_raloe_crono

Esto cumple una regla importante del proyecto:

No queremos romper la comunicacion con el cliente devolviendo errores innecesarios.

Preferimos devolver una respuesta controlada que indique que el PDF no corresponde al perfil esperado.

8.9. Contrato vacio

Cuando el PDF si parece Raloe-CRONO, el sistema parte de un contrato vacio.

Este contrato contiene todas las secciones y todos los campos que esperamos devolver.

Ejemplos de secciones:

- general
- Normas
- Caracteristicas
- Traccion_electrica
- Traccion_hidraulica
- Puertas_cabina_embarque_1
- Puertas_cabina_embarque_2
- Gestion_foso_huida_reducida
- Opciones
- Premontada
- Armario
- Botonera_Exterior
- Botonera_Cabina

La ventaja de partir de un contrato vacio es que el JSON final siempre tiene la misma forma.

Si una seccion no aparece en el PDF, sus campos quedan vacios.

Si un campo concreto no se encuentra, queda vacio.

Esto facilita mucho el consumo desde el cliente porque evita tener que preguntar continuamente si una clave existe o no existe.

8.10. Extraccion de observaciones

Las observaciones son un caso especial.

No pertenecen a las paginas tecnicas principales.

Se buscan en la pagina resumen, que normalmente es la pagina 1.

El extractor correspondiente es:

ExtractorObservacionesResumenRaloeCrono

Si hay observaciones, se devuelve el texto.

Si no hay observaciones, se devuelve vacio.

No se espera un aviso en el PDF.

El sistema debe mirar siempre si existen observaciones o no.

8.11. Deteccion de paginas importantes

El sistema no asume que las paginas importantes sean siempre las tres ultimas.

Esto es muy importante.

En los PDFs actuales hemos visto que:

En el PDF 654391:

La pagina tecnica principal esta en la pagina 5. Foso, huida y opciones estan en la pagina 6. Premontada, armario y botoneras estan en la pagina 7.

En el PDF 654340:

La pagina tecnica principal esta en la pagina 4. Foso, huida y opciones estan en la pagina 5. Premontada, armario y botoneras estan en la pagina 6.

Por eso el sistema usa detectores de pagina.

Los detectores estan en:

`src/extractor_pdf/infrastructure/selection/detectores_paginas_raloe_crono.py`

Actualmente existen:

DetectorPaginaTecnicaRaloeCrono

Busca la pagina con informacion como Serie, Normas, Caracteristicas, Traccion Electrica y Puertas de Cabina.

DetectorPaginaFosoHuidaOpcionesRaloeCrono

Busca la pagina con Gestion foso / huida reducida y Limitador velocidad Cab.

DetectorPaginaPremontadaArmarioBotonerasRaloeCrono

Busca la pagina con Premontada y Botonera Cabina.

Cada detector revisa el texto de las paginas y decide cual encaja mejor.

8.12. Extraccion de la pagina tecnica principal

Una vez detectada la pagina tecnica principal, se ejecuta:

`ExtractorPaginaTecnicaRaloeCrono`

Este extractor obtiene secciones como:

- general
- Normas
- Caracteristicas
- Traccion_electrica
- Traccion_hidraulica
- Puertas_cabina_embarque_1
- Puertas_cabina_embarque_2

Aqui aparecen datos como:

- Serie
- Pais_instalacion
- Idioma_documentacion
- Maniobra
- Tipo
- Arq

- Velocidad
- Paradas
- Fabricante_motor
- Modelo_motor
- Variador
- Puertas de cabina

Tambien se tratan campos tipo check.

En el PDF visualmente pueden aparecer como casillas marcadas o no marcadas.

En el JSON se devuelven como:

Si

No

De esta manera el cliente no tiene que interpretar la representacion visual del PDF.

Recibe directamente un valor normalizado.

8.13. Extraccion de foso, huida y opciones

Despues se detecta y extrae la pagina de:

Gestion foso / huida reducida

Opciones

El extractor es:

ExtractorFosoHuidaOpcionesRaloeCrono

Esta pagina contiene muchos campos booleanos y muchos campos opcionales.

Ejemplos:

- Foso_exencion_raloe
- Huida_EN81_21
- Limitador_velocidad_cab
- Pesacargas_fabricante
- Rescate
- Luz_en_armario
- Escalera_foso_con_contacto
- Magnetotermicos_y_diferenciales
- Test_de_freno
- Modulo_ARM
- Modulo_DCI

Si un campo no esta informado, se devuelve como cadena vacia.

Si un check esta marcado o no marcado, se devuelve como Si o No.

8.14. Extraccion de premontada, armario y botoneras

La tercera pagina tecnica contiene:

- Premontada

- Armario
- Botonera_Exterior
- Botonera_Cabina

El extractor es:

ExtractorPremontadaArmarioBotonerasRaloeCrono

Esta pagina tiene algunos campos especiales porque representan listas.

Ejemplos:

Piso E1

Accesso_E1

Display_modelo_E1

Secuencia

Para la familia CRONO hay un limite de 32 plantas.

Por eso algunos campos deben devolver 32 posiciones.

Ejemplo:

A,A,A,A,A,-

En cambio, Distancia_niveles representa distancias entre plantas.

Si hay 32 plantas, como maximo puede haber 31 distancias entre ellas.

Por eso `Distancia_niveles` debe devolver hasta 31 posiciones.

Esta normalizacion es importante para que el cliente reciba siempre la misma estructura.

8.15. Union de resultados

Cada extractor devuelve una parte del JSON.

El caso de uso va uniendo esas partes dentro del contrato completo.

El resultado final se devuelve dentro de:

data

Ademas, para depuracion y trazabilidad, tambien se devuelven algunas paginas separadas:

page 1

page 4

page 5

page_6

page_7

El numero exacto depende del PDF.

Por ejemplo, si la pagina tecnica principal se detecta en la pagina 4, aparecera:

page_4

Si se detecta en la pagina 5, aparecera:

page 5

Esto ayuda a revisar de donde han salido los datos.

8.16. Campos extra

El sistema tambien contempla una seccion de campos extra.

La idea es detectar informacion que aparece en el PDF pero que no forma parte del contrato esperado.

Esto es util cuando el cliente cambia el formulario.

Ejemplo:

- anade un campo nuevo;
- cambia una etiqueta;
- elimina una seccion;
- mueve informacion de sitio.

En vez de perder completamente esa informacion, el sistema puede devolverla en una seccion adicional.

Esto permite detectar cambios de formulario y decidir si hay que actualizar el perfil de extraccion.

Es una forma de convertir cambios silenciosos del PDF en algo visible para el equipo.

8.17. Metadatos de respuesta

Ademas de data, la respuesta incluye metadata.

metadata contiene informacion tecnica de la extraccion.

Ejemplos:

filename

Nombre del archivo recibido.

profile_id

Perfil usado para extraer.

is_raloe_crono

Indica si el PDF parece pertenecer al perfil Raloe-CRONO.

status

Estado de la extraccion.

warnings

Lista de avisos no bloqueantes.

technical_page

Pagina donde se encontro la pagina tecnica principal.

pit_escape_options_page

Pagina donde se encontro foso, huida y opciones.

premounted_cabinet_buttons_page

Pagina donde se encontro premontada, armario y botoneras.

Los metadatos no son los datos de negocio principales.

Sirven para diagnostico, soporte y trazabilidad.

8.18. Respuesta JSON final

La respuesta final tiene esta forma general:

```
{
  "data": { "...": "..."}, "page_1": { "Observaciones": "" }, "page_5": { "...": "..."}, "metadata": { "filename": "654391.pdf",
    "profile_id": "raloe_crono", "is_raloe_crono": true, "status": "ok", "warnings": [] }
}
```

La parte mas importante para el cliente sera normalmente:

data

Porque data contiene el contrato completo ya normalizado.

8.19. Que ocurre si falta una seccion

Una premisa importante del proyecto es:

No queremos devolver error simplemente porque falte una seccion.

Si falta una seccion, sus campos se devuelven vacios.

Si falta un campo, ese campo se devuelve vacio.

Si no se encuentra nada que permita identificar el PDF como Raloe-CRONO, entonces se devuelve:

is_raloe_crono: false

status: not_raloe_crono

Esto permite que el sistema consumidor pueda actuar de forma controlada.

8.20. Pruebas automaticas

Las pruebas automaticas se ejecutan con pytest.

Desde la terminal del proyecto:

```
pytest
```

Si hay problemas con el entorno virtual o con las rutas, tambien se puede ejecutar indicando Python directamente:

```
python -m pytest
```

Las pruebas comprueban, entre otras cosas:

- lectura de paginas del PDF;
- deteccion de la pagina tecnica;
- deteccion de la pagina de foso, huida y opciones;
- deteccion de la pagina de premontada, armario y botoneras;
- extraccion de campos por pagina;
- comparacion contra JSON esperado;
- llamada real al endpoint /extract usando TestClient.

Los JSON esperados estan en:

tests/fixtures/expected

Estos ficheros son importantes porque representan la verdad esperada del sistema.

Cuando un test compara la extraccion contra uno de esos JSON, esta comprobando que el extractor devuelve exactamente lo esperado.